

Discussion

For this Ben Olschar, 108021211678 and Frederik Hüttemann, 108021215247 cooperated with each other.

We did implement the code on our own, but agreed to using one version. The discussion and results are made in cooperation.

Task 1: CPU Implementation

Obviously, the CPU implementation is pretty slow as it does all calculations sequentially.

Task 2: GPU implementation with usage of global memory

This implementation is kind of the comparison between the CPU and GPU implementation as both rely on accessing the global memory. The GPU however is ~2400 times faster than the CPU as it can perform the operations in parallel.

Task 3: GPU implementation with constant memory for the kernel mask

This implementation tries to optimize the baseline GPU implementation from task 2. Instead of using global memory for the kernel mask, we use constant memory for it. This results in again a speedup of ~1.5 times than before. Before, the memory access on the kernel was not optimal as many threads from different warps tried to access the same memory address which results in memory conflicts. Using constant memory will result in the GPU caching (texture cache) the mask entries. This allows faster memory access in all warps. Therefore, the speedup is expected.

Task 4: GPU implementation with constant memory for the kernel mask and texture memory for the image

Here, we want to even further improve the implementation using texture memory for the image. We use `cudaMemcpy2DToArray` as the function `cudaMemcpyToArray` from the lecture is deprecated. Again, we expect a speedup as this kind of memory optimizes caching on 2D locations. In the previous homework we have seen that accessing different rows of an image is slow in one dimensional memory. As we perform flattening on our images, the previous implementation suffers from this effect as image entries, which lay right below each other, are stored far away in memory. Using texture memory, this is optimized. For interpolation, we do have this accessing and therefore expect a speedup when storing the image in 2D texture memory.

When looking at the results, we do observe a speedup about 3 percent. This might be explained by regular automatic caching of the global memory working just as good as in the other runs. To be honest, this speedup is less than we expected and was pretty disappointing, because we expected more optimal memory access.